Exercise 2 – NLP Text Preprocessing

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract*— This article describes the use of a number of preprocessing techniques on a real-world dataset of tweets labeled for hate speech and other offensive language. The main goal was to clean the text and standardize the data for downstream natural language processing (NLP). Python and libaries such as pandas, NLTK and regular expressions were used to follow a sequence of processing steps, such as lower-casing the text, expanding contractions and removing usernames, URLs, numbers, special characters and stopwords. Repeated-character normalizations were also utilized to help normalize the noise often present in user-generated content. The cleaning process in total reduced the initial word count of the dataset from 349,862 to 199,158, emphasizing how much material that was non-value-added was removed. The actual material transformed the semantic clarity and consistency of the dataset in a way that was more conducive to reliable NLP analysis. In short, this exercise has demonstrated a strong case for the value of good preprocessing to improve data quality or prepare text in a form that that was conducive to a more meaningful computational interpretation.**

Keywords— text preprocessing, NLP, hate speech detection, Twitter dataset, tokenization, stopword removal

I. Introduction

In any NLP (Natural Language Processing) pipeline, text preprocessing is a critical preliminary process. Raw text is often rather noisy. Noise can include how we speak and spell things (slang, misspellings, casing, and other punctuation-ridden artifacts), or the existence of special symbols, most likely if raw text includes social media data. Noisy text can significantly ruin the quality of NLP work that follows, most importantly related tasks classification, sentiment analysis, and topic modeling. The text preprocessing step procedures raw unstructured text and formats it before analyzing as it cleans the unstructured text to display more of a normalized format.

The main purpose of this activity is to identify, capture, and cleanse an unstructured text data set using various preprocessing techniques. This task is meaningful to allow students to understand the challenges of handling raw textual (real) data, regardless of the task at hand, but especially in regard to hate speech or offensive language detection. The students will gain hands-on experience and knowledge of tokenization, normalization, lemmatization, and noise removal, which are all components of regular engaged practice and, even more, potential NLP applications.

This paper outlines the methods used in preprocessing, presents observations from the dataset, and provides examples comparing the raw and cleaned text. The remainder of this paper is organized as follows: Section II describes the dataset and preprocessing functions, Section III explains the preprocessing steps, Section IV presents results and visualizations, and Section V discusses the paper's findings, and Section VI concludes the exercise.

II. Related Works

Text preprocessing is a fundamental step in natural language processing (NLP) workflows that connects the raw text data and final applications such as hate speech detection. There are many studies that emphasize that the quality and relevance of the results in downstream applications were influenced by how the data was cleaned or structured. Each data preprocessing technique, from lowercasing text to tokenizing sentences, helps minimize noise, so machine-learning models can aid in recognizing relevant linguistic patterns. In the following subsections, each the relevance of each preprocessing step is discussed, as well as literature and research that justifies their use in this activity.

a) Lowercasing

Lowercasing text is an important process in natural language processing (NLP) to help provide consistency across data. Lowercasing text means taking every character of a textual dataset and changing it to downcase. This provides one less variable to account for with respect to inconsistencies caused by case and could improve later NLP tasks like tokenizing and removing stopword. Lowercase provides a level of simplification to the vocabulary of words because ties three different tokenization options ("Apple", "apple", and "APPLE") into one token. This lessens the complexity of tokens in the dataset and thus, allows models to be more productive. The lowercasing can be performed fairly easily in Python usually by using the built-in lower() string method for a single string or a list of texts. Lowercasing makes it easier for case to be unified, and produces cleaner and more uniform data that is important for any natural language understanding and machine learning processes. [2]

b) Contractions Expansion

An important preprocessing step in natural language processing (NLP) is expanding contractions; taking abbreviated forms, such as "can't" or "won't", and replacing

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

them with full-forms, "cannot" and "will not". When processing text in NLP, normalizing text reduces the variation in the text, and may help improve performance in downstream NLP tasks such as hate speech detection. [3]

There are some libraries in Python including contractions or pycontractions that allow you to expand some contractions. The contractions library is very simple and it will identify contractions in a text and produce the expanded text while preserving the original punctuation and spacing. This is useful, especially with casual text data that may include contractions, because contractions are so common and may significantly change the accuracy of any analyses.

c) Special Characters Removal

Removing special characters is one of the most important steps of preprocessing in natural language processing (NLP). This includes punctuation, symbols, and other non-alphanumeric characters. Removing special characters helps ensure quality of the input data by removing noise, as noise can hinder analysis and ultimately performance in any NLP task such as text classification and sentiment analysis.

In Python, special character removal can be accomplished with regular expressions (regex) or string manipulation. Libraries (re) can help in finding and removing characters. The removal of special characters means that text can be cleaned from unwanted characters and prepared for subsequent operations. However, data analysts should also consider the analysis context since some punctuation marks may have semantic meaning behind them, removing them can lead to failure to include important meaning, for example, "Johnny loved Paul's cake!" to "Johnny loved Pauls cake". [4]

Removing special characters can help create cleaner and consistent datasets for analysis in NLP which ultimately leads to better performance.

d) Stop Words Removal

Eliminating stop words is a valuable preprocessing step in natural language processing (NLP) and is especially beneficial for finding hate speech. Stop words are common, uninformative words, like "and," "the," and "is," that do not generally add value to the textual meaning, while serving only to add noise to the analysis. Sorting out stop words allows focus on the more informative words and in total sums up the text better, which is typically a primary goal of all textual analysis.

Python has utilities such as NLTK, spaCy, and Gensim that make it easy and straightforward to eliminate stop words from text data. This will not only give dimensionality reduction to the dataset, but will also allow machine learning models to focus on more meaningful features, making their performance optimal. In the case of hate speech detection specifically, removing stop words helps treat data while isolating the target language, boosting model performance. [5]

In most cases, removing stop words is positive, but it is important to think about the context surrounding the analysis. Based on the texts, data analysts may want to keep certain stop words that carry relevant and contextual meaning. Therefore, careful consideration should be given to the stop words chosen for removal based on the given requirements of the NLP task at hand.

e) Tokenization

Tokenization is a key precursor step in natural language processing (NLP) text that describes a process of segmenting text into smaller parts called tokens. Tokens may be single words, phrases, or even characters, depending on the analysis. Tokenization is required for NLP tasks including hate speech detection because it allows for further detailed aspects for consideration from analysis, and enables the model to observe patterns and relationships amongst words.

Python has various libraries to provide useful ways to accomplish tokenization without it being too laborious including NLTK, spaCy, and the tokenizers library. All these documentations will provide different methods of tokenization, such as word tokenization, sentence tokenization, subword tokenization so that data analysts can use whichever is most suitable for the degree of tokenization they want to create. For example, word tokenization involves splitting the text into single words, sentence tokenization divides the text into sentences which can be useful to analyze text before the analysis to understand the text structure. [6]

Effective tokenization not only breaks up the text, but also makes it easier to follow extraction steps like removing stop words or applying stemming and lemmatization. When text is broken into tokens, data analysis allows analyst to represent the data in structured ways that are more suitable for use in machine learning algorithms.

f) Lemmatization

In natural language processing (NLP), lemmatization is a valuable technique of text normalization which reduces words to their base or lemma forms. Stemming tends to artificially truncate a word without any measure of context, while lemmatization uses context and part of speech (POS) to reduce inflected words into their meaningful root forms correctly. In other words, lemmatization combines inflected words into a single lemma, "run" for instance which can be used to describe "running", "ran", and "runs", which is desirable in text analysis and NLP as it reduces the number of words and improves the quality of analysis.

There are a number of NLP functions that utilize lemmatization including information retrieval, sentiment analysis, and more. Lemmatization is done to improve how users understand language and information and also to reduce the amount of noise in the data when text processing. Lemmatization uses dictionaries and

morphological analysis to arrive at the most correct meaning for the word in context. [7]

Lemmatization is an important preprocessing task that enables better extraction of feature and yields better natural language understanding quality when used in modern NLP pipelines.

Preprocessing text efficiently with techniques such as lowercasing, contractions expansion, special characters removal, stop words removal, tokenization, and lemmatization is a critical part of improving data quality and model efficacy in an NLP task such as hate speech detection. Each step diminishes noise and standardizes the text so it can be analyzed with more fidelity.

III. METHODOLOGY

Content Warning:

The following screenshots may contain uncensored examples of offensive and hate speech from a real-world dataset used for academic purposes. The content has not been altered to preserve the integrity of preprocessing analysis.

a) Dataset Description

The dataset utilized in this practice is the Hate Speech and Offensive Language Dataset published by Davidson et al. (2017), and which is hosted publicly on GitHub. The dataset was compiled for their study "Automated Hate Speech Detection and the Problem of Offensive Language," presented at the 11th International AAAI Conference on Web and Social Media (ICWSM 2017).

This dataset contains 24,783 rows and 6 columns, found in a CSV file format. The dataset is written in English and was manually annotated by CrowdFlower contributors. Each row corresponds to a tweet and bears the tweet text along with annotations. The data is an estimated 349,862 words in size, making it a valuable resource to study noisy and informal language from social media.

The dataset consists of six key columns:

- tweet_id -- tweet unique ID
- tweet - tweet text
- class - class label of majority class (0 = hate speech, 1 = offensive language, 2 = neither)
- hate_speech - number of CrowdFlower users who labelled it as hate speech.
- offensive_language - number of users labelled it offensive.
- neither - number of users labelled as neither.

The researchers request that their work be cited in any published research that uses this dataset. [8]

b) Existing Noise

When looking at the raw tweets, a number of text noise was observed that will adversely affect downstream NLP analysis:

- Inconsistent case: Tweets feature both capitalized and non-capitalized letters, making token matching difficult.
- Contractions (e.g. don't, I'm): These are often expanded for consistency and clarity.
- Special characters and symbols: Tweets feature hashtags (#), mentions (@), emoji, and punctuation (e.g. !!!) that do not provide semantic meaning for many NLP tasks.
- Stop words: Common words (e.g. the, is, and) that have little informational value when doing, say, classification or sentiment analysis.
- Token irregularities: Tweets may have clumped words, URLs, or repeated letters (e.g. lots of letters all jumbled together) that will make tokenization difficult.
- New type of noise: A new form of noise identified in this data set were repeated, single letter words (e.g. “a a a a”) and character spam (e.g. “loooool”, “yessss”), which were not explicitly indicated in lecture but occurred often in the tweet data. Specific clean-up rules needed to be created for these.

These noise types highlight the need for a strong preprocessing pipeline that is specific to the inherently informal and chaotic nature of social media text.

c) Preprocessing Proper

I.) Libraries Installation

```
!pip install contractions
!pip install spacy
!python -m spacy download en_core_web_sm
```

Fig.1. Python Libraries Installation

This cell installs the required Python libraries needed for the text preprocessing. Each of these libraries serves a different purpose when it comes to cleaning and analyzing textual data.

The contractions library expands contractions, such as “don’t” to “do not”, which can help to facilitate greater consistency and accuracy during the later processing steps such as tokenization and lemmatization. Spacy is a

powerful NLP library providing lots of useful tools for tokenization, part-of-speech tagging and lemmatization. The final line below downloads a small-en language model, en_core_web_sm, which contains the dictionaries and rules for processing English text.

This is the initial set-up and ensures that you have all the required tools installed before you start the pre-processing of the tweets.

```
import pandas as pd
import numpy as np
import re
import nltk
import html
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import contractions
from nltk.tokenize import TweetTokenizer
import spacy
from nltk.corpus import wordnet
from nltk import pos_tag
import matplotlib.pyplot as plt


nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

Fig. 2. Python Libraries Imports

The cell imports all of the necessary Python libraries and modules needed for the preprocessing workflow. It imports the libraries for working with structured data, the regex and html libraries for cleaning tweets, the Natural Language Toolkit (NLTK) tokenizers, the stopwords, and the WordNet lemmatzier, which are needed for breaking tweets apart into words and reducing them to a word's root form. It utilizes a contractions package to normalize informal or mashup language. The TweetTokenizer is also a tokenizer that is more tuned towards the needs of social media text processing.

The spacy and pos_tag tools are imported to help make better decisions about identifying grammatical roles, which assists the lemmatization process later on.

Finally, the nltk.download() commands are used to make sure that we have all of the datasets available locally, especially the stopword lists and WordNet dictionaries, along with any pre-trained models for tokenizers and part-of-speech tagging. None of the functions used later will work without downloading these files.

II.) Dataset Import

Fig. 3. Dataset Reading

This cell loads the dataset and displays the first few entries. The read_csv() function reads in the labeled_data.csv file which contains tweets labeled for hate speech and offensive language. The dataset includes metadata, such as a count of the annotators and their judgments, and the tweet text.

The pd.set_option('display.max_colwidth', None) line ensures that when displaying a DataFrame, long tweets are not truncated. Finally, df.head(10) prints the first ten rows of the data so that we can see the raw structure of the data and its content, before we do any cleaning. This step gives us an idea of what kind of preprocessing we may need to do.

```
patterns = {
    'Usernames (@user)': r'@\w+',
    'URLs (http/https/www)': r'http\S+|www\S+|https\S+',
    'Numbers': r'\b\d+\b',
    'Special Characters': r'[^A-Za-z0-9\s]'
}

noise_counts_before = {label: df['tweet'].str.count(pattern).sum() for label, pattern in patterns.items()}

total_words_before = df['tweet'].str.split().apply(len).sum()
noise_counts_before['Total Words'] = total_words_before

plt.figure(figsize=(8,5))
bars = plt.bar(noise_counts_before.keys(), noise_counts_before.values(), color='skyblue')
plt.title('Noise Frequency and Total Word Count Before Cleaning')
plt.ylabel('Total Occurrences')
plt.xticks(rotation=25)

for bar in bars:
    height = bar.get_height()
    plt.annotate(f'{int(height):,}',
                 xy=(bar.get_x() + bar.get_width() / 2, height),
                 xytext=(0, 3),
                 textcoords='offset points',
                 ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

Fig. 4. Noise Frequency and Word Count Before Cleaning Code

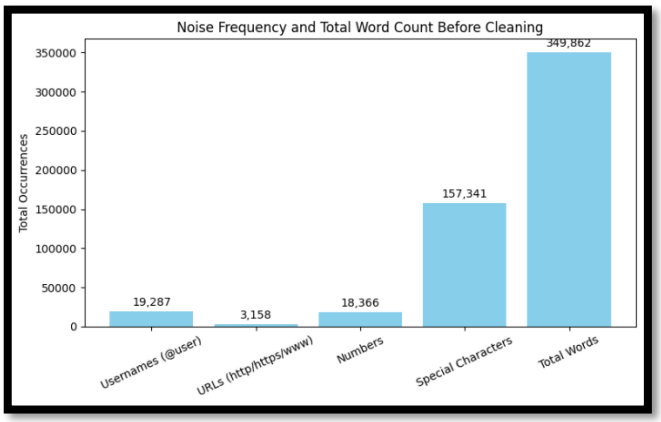

Fig. 5. Fig. 3 Output

The code takes counts of common noise types in the tweet texts by matching specific patterns using regular expressions for noise types, which include usernames, URLs, numbers, and special characters. The code counts total matches for each of the noise types across all tweets.

Then, the counts are presented in a bar chart to provide a visual representation of how much noise is present. Running this on the original raw tweets shows the noise level of the initial dataset prior to cleaning.

III.) Lowercasing the Dataset

```
testcase_text = df.loc[3, 'tweet']
print(testcase_text)

!!!!!!!!! RT @C_G_Anderson: @viva_based she look like a tranny
```

Fig. 6. Row Preview Code

One tweet was extracted from the dataset based on its row index to use as a preview of the preprocessing steps. It is a tweet that has typical social media elements like lots of exclamation points, mentions to users, informal language, and possibly offensive content. This sample will be used to show how each preprocessing function will transform raw text into a cleaner, more structured text.

```
lowercased_test = testcase_text.lower()
print( lowercased_test)

!!!!!!!!! rt @c_g_anderson: @viva_based she look like a tranny
```

Fig. 7. Lowercased Row Code

Selected tweet was converted to lower case using the (.lower()) function. This step was executed to maintain consistency and remove case sensitivity. Lowercasing tweets is a standard preprocessing step in NLP, as it addresses issue of treating "She" and "she" as three separate words. Structurally, the text remains the same except that all characters are lower.

```
df['tweet'] = df['tweet'].apply(lambda x: str(x).lower())

df[['tweet']].head(10)
```

	tweet
0	!!! rt @mayasolovely: as a woman you shouldn't complain about cleaning up your house. & as a man you should always take the trash out...
1	!!!!! rt @mleew17: boy dats cold...tyga dwn bad for cuffin dat hoe in the 1st place!!
2	!!!!!!! rt @urkindofbrand dawg!!!! rt @80sbaby4life: you ever fuck a bitch and she start to cry? you be confused as shit
3	!!!!!!!!! rt @c_g_anderson: @viva_based she look like a tranny
4	!!!!!!!!!!!! rt @shenikaroberts: the shit you hear about me might be true or it might be faker than the bitch who told it to ya 
5	!!!!!!!!!!!!!!!"@t_madison_x: the shit just blows me..claim you so faithful and down for somebody but still fucking with hoes! 😂😂😂"
6	!!!!!!"@__brighterdays: i can not just sit up and hate on another bitch .. i got too much shit going on!"
7	!!!!“@selfiequeenbri: because i am tired of you big bitches coming for us skinny girls!!”
8	" & you might not get ya bitch back & that is that "
9	" @rhythmixx_ :hobbies include: fighting mariam"\n\nbitch

Fig. 8. Lowercased Dataset Code

Using the .apply() function with a lambda expression, we converted all tweets in the dataset to lowercase. It is a good idea to work only with strings that are lowercase when examining individual entries in the 'tweet' column. Lowercasing the tweets allows us to eliminate inconsistencies resulting from different capitalizations of the same word, preventing unwanted mistakes during tokenization, comparisons, and analysis. A snapshot of the first 10 lowercase tweets shows that all lowercase works as intended.

IV.) Contractions Expansion

```
def expand_contractions(text):
    return contractions.fix(text)

df['tweet'] = df['tweet'].apply(expand_contractions)

pd.set_option('display.max_colwidth', None)

df[['tweet']].head(10)
```

	tweet
0	!!! rt @mayasolovely: as a woman you should not complain about cleaning up your house. & as a man you should always take the trash out...
1	!!!!! rt @mleew17: boy dats cold...tyga dwn bad for cuffin dat hoe in the 1st place!!
2	!!!!!!! rt @urkindofbrand dawg!!!! rt @80sbaby4life: you ever fuck a bitch and she start to cry? you be confused as shit
3	!!!!!!!!! rt @c_g_anderson: @viva_based she look like a tranny
4	!!!!!!!!!!!! rt @shenikaroberts: the shit you hear about me might be true or it might be faker than the bitch who told it to ya 
5	!!!!!!!!!!!!!!!"@t_madison_x: the shit just blows me..claim you so faithful and down for somebody but still fucking with hoes! 😂😂😂"
6	!!!!!!"@__brighterdays: i can not just sit up and hate on another bitch .. i got too much shit going on!"
7	!!!!“@selfiequeenbri: because i am tired of you big bitches coming for us skinny girls!!”
8	" & you might not get ya bitch back & that is that "
9	" @rhythmixx_ :hobbies include: fighting mariam"\n\nbitch

Fig. 9. Contractions Expansion Code and Output

The contractions found in tweets were expanded using the contractions library. The library has a function that applies contractions.fix() to each tweet replacing shortened forms like you're or it's with their full version you are or it is. This is helpful when wanting to read the text, especially for later processing such as tokenization or classification. Finally, the output confirms that contractions were expanded and this improved consistency within the dataset.

V.) Special Characters Removal

1) Regular Expression (RegEx) removal

```
test_case = df['tweet'][0]
print("Before cleaning:", test_case)

Before cleaning: !!! rt @mayasolovely: as a woman you should not complain about cleaning up your house. &amp;amp; as a m
```

Fig. 10. Before RegEx Removal Code

A sample tweet from the dataset was shown to demonstrate its status after a lowercase transformation and an abbreviation expansion. This tweet still contains duplicate spaces, unwanted characters, and some HTML encodings like & that will be removed during the remaining cleaning steps in preparation for text analysis.

```
test_case = re.sub("[^a-zA-Z0-9\s]", '', test_case)
print("After Regex Cleaning:", test_case)

After Regex Cleaning:  rt mayasolovely as a woman you should not complain about cleaning up your house amp as a man
```

Fig. 11. After Regex Removal Code and Output

This phase employs a regex to remove from the tweet all characters except letters, digits, and whitespaces. Punctuation and special characters such as exclamation signs (!), colons (:), or HTML encodings (& converts to amp) would be eliminated. The outcome is a tidier and uniform version of the tweet that's readily tokenizable and analyzable in downstream tasks, such as hate speech detection.

```
df['tweet'] = df['tweet'].apply(lambda x: re.sub(r'\d+', '', x))
```

Fig. 12. Numeric Digits Removal Code

Another regular expression was applied to remove all numeric digits from the tweets. This is done to ensure that only meaningful text remains for processing.

```
print(df['tweet'][0])
rt mayasolovely as a woman you should not complain about cleaning up your house amp as a man you should always take
```

Fig. 13. Row Output to Show RegEx Removal Output

The dataset produced a cleaned tweet to examine the results. The output shows that the tweet now contains a cleaned input which includes only lowercase letters and whitespace. Thus, we know that unwanted characters and numbers have been removed.

2) @Username Mentions Anonymization

```
df['tweet'] = df['tweet'].apply(lambda x: re.sub(r'@\w+', '<USER>', x))
```

Fig. 14. Usernames Anonymization

All usernames starting with @ are replaced with the placeholder to anonymize the data and remove personal mentions that do not contribute to sentiment or hate speech analysis.

3) URLs Removal

```
df['tweet'] = df['tweet'].apply(lambda x: re.sub(r'http\S+|www\S+|https\S+', '', x))
```

Fig. 15. URLs Removal Code

All URLs beginning with http, https, or www are removed from the tweets to clean up links that may distract from the main textual content.

4) Numbers Removal

```
whitelist_numbers = {'911', '100', '420', '666', '69'}  # Add more as needed

def remove_non_whitelisted_numbers(text):
    return re.sub(r'\b\d+\b', lambda m: m.group() if m.group() in whitelist_numbers else '', text)

df['tweet'] = df['tweet'].apply(remove_non_whitelisted_numbers)
```

Fig. 16. Numbers Removal Code (Except Whitelisted Numbers)

In this step, all duplicate standalone numbers, except those in the defined whitelist (e.g., 911, 420, 666), are removed from the tweets. These whitelisted numbers are kept because they might be culturally or contextually relevant in an analysis of text.

5) HTML Entities Removal

```
df['tweet'] = df['tweet'].apply(lambda x: html.unescape(x))

df['tweet'] = df['tweet'].str.replace(r'^rt\s+', '', regex=True)
```

Fig. 17. HTML Entities Removal Code

This step performs two cleaning actions on one cell. First it will, convert any HTML entities contained in tweets (ex: &), into a character it represents, such as &, via the html.unescape() function. This will help in understanding the content and make processing the text a little easier. Second, it will remove the "rt" designation at the beginning of tweets, which stands for retweet! This is done through a regular expression strictly targeting "rt" at the beginning of a string, which will help to refine the tweet content and remove extraneous data.

6) Non-alphanumeric Characters Removal

```
df['tweet'] = df['tweet'].apply(lambda x: re.sub(r'[^A-Za-z0-9\s.,!?\'\"]', '', x))
```

Fig. 18. Non-alphanumeric Characters Removal Code

The above line strips away all characters from each tweet except alphabets, numbers, spaces, and basic punctuation: period, comma, exclamation point, question mark, single quotation mark, double quotation mark. By limiting the characters to the selected characters, the tweet text is more consistent and cleaner by minimizing variations in the tweets, while still keeping useful punctuation that may convey meaning or tone.

7) Extra Spaces Removal

```
df['tweet'] = df['tweet'].apply(lambda x: re.sub(r'\s+', ' ', x).strip())
```

Fig. 19. Extra Spaces Removal Code

This cell eliminates excess spaces in tweet text by replacing instances of multiple whitespace characters with a single whitespace character and removing lead or trailing spaces. Whitespace normalization leads to a more coherent text structure.

8) Post-Special Characters Removal Output

```
pd.set_option('display.max_colwidth', None)
df[['tweet']].head(10)
```

Fig. 20. First 10 Dataset Entries Code

	tweet
0	rt mayasolovely as a woman you should not complain about cleaning up your house amp as a man you should always take the trash out
1	rt mleew boy dats coldtyga dwn bad for cuffin dat hoe in the st place
2	rt urkindofbrand dawg rt sbabylife you ever fuck a bitch and she start to cry you be confused as shit
3	rt cganderson vivabased she look like a tranny
4	rt shenikaroberts the shit you hear about me might be true or it might be faker than the bitch who told it to ya
5	tmadisonx the shit just blows meclaim you so faithful and down for somebody but still fucking with hoes
6	brighterdays i can not just sit up and hate on another bitch i got too much shit going on
7	selfiequeenbri because i am tired of you big bitches coming for us skinny girls
8	amp you might not get ya bitch back amp that is that
9	rhythmbox hobbies include fighting mariam bitch

Fig. 21. Post-Special Characters Removal Output

This cell adjusts the display so that the full text of tweets appears without being cut off, and then shows the first 10 cleaned tweets to visualize the preprocessing results. The following procedures have already been applied:

- Regular expression character cleaning:
- Removing @username mentions.
- Removing tweets with URLs (http, https, www links).
- Removing numbers (except for whitelisted ones like 911; 420, etc.)
- Converting HTML entities in tweets to readable text (converted & to & etc.)
- Removing anything that is not an alphanumeric character (excluding the allowed punctuation).
- Removing excess whitespace.

VI.) Stop Words Removal

```
stop_words = stopwords.words('english')
print(stop_words)

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'a
```

Fig. 22. English Stop words List

This cell loads the standard list of English stop words from the NLTK library. Stop words are common words like the, is, in, and, of that are usually filtered out in natural language processing tasks, as they are not very meaningful content. The stop words list includes contractions and variations like don't, you're, and couldn't that will help reduce grammatical noise before continuing on with text analysis.

```
test_case = df['tweet'][0]
print(test_case)

final_test = []
for word in test_case.split():
    if word == '' or '\r\n' in word or word in stop_words:
        None
    else:
        final_test.append(word)

print(final_test)
print(' '.join(final_test))

rt mayasolovely as a woman you should not complain about cleaning up your house amp as a man you should always take
['rt', 'mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'amp', 'man', 'always', 'take', 'trash']
rt mayasolovely woman complain cleaning house amp man always take trash
```

Fig. 23. Cleaning Tokens by Removing Stop words and Irregular Words

This cell shows the process of manually filtering out stop words from a tweet. A random tweet from the dataset is chosen and it is tokenized by splitting it into words. Each word is checked and if it is a stop word, an empty string or contains newlines, it is skipped. If not, it is added to a new list. The end result is a cleaned version of the tweet that has retained the meaningful content for analysis.

```
def remove_stopwords(article, stopwords):
    temp_fnl = []
    for word in article.split():
        if word == '' or '\r\n' in word or word in stopwords:
            None
        else:
            temp_fnl.append(word)
    return ' '.join(temp_fnl)


remove_stopwords(test_case, stop_words)

'rt mayasolovely woman complain cleaning house amp man always take trash'
```

Fig. 24. Custom Function for Stopword Removal and Token Filtering

In this cell we define a remove_stopwords() function that removes common stopwords from a supplied string. The remove_stopwords() function operates by first splitting the text input into words. Then it checks each word against the supplied stopwords list, and if there is a match it does not include it (and ignores any blank strings or strings with newline characters). Finally, the words that we did end up keeping are combined back into a cleaned string. The function is then tested on a sample tweet and successfully removes the stopwords leaving only the words of value.

```
df['tweet'] = df['tweet'].apply(lambda x: remove_stopwords(x, stop_words))

pd.set_option('display.max_colwidth', None)
df[['tweet']].head(10)
```

	tweet
0	rt mayasolovely woman complain cleaning house amp man always take trash
1	rt mleew boy dats coldtyga dwn bad cuffin dat hoe st place
2	rt urkindofbrand dawg rt sbabylife ever fuck bitch start cry confused shit
3	rt cganderson vivabased look like tranny
4	rt shenikaroberts shit hear might true might faker bitch told ya
5	tmadisonx shit blows meclaim faithful somebody still fucking hoes
6	brighterdays sit hate another bitch got much shit going
7	selfiequeenbri tired big bitches coming us skinny girls
8	amp might get ya bitch back amp
9	rhythmixx hobbies include fighting mariam bitch

Fig. 25. Dataset Output After Stopwords Removal

This cell uses the custom stopword removal function and applies it to each tweet in the dataset. The apply() method is utilized to loop through each row in the 'tweet' column and execute the remove_stopwords() function, which utilizes the NLTK stopword list to remove frequently used but non-informative words.

The line pd.set_option('display.max_colwidth', None) enables us to see the entire content of each tweet without it being cut off when we display the cleaned tweets. The last line, df[['tweet']].head(10), displays the first ten tweets (after we have processed them) to check what the text of the tweets look like after stopword removal.

VII.) Tokenization

```
from nltk.tokenize import word_tokenize

test_case = df['tweet'][0]
print("Original:", test_case)

tokens = word_tokenize(test_case)
print("Tokens:", tokens)

Original: rt mayasolovely woman complain cleaning house amp man always take trash
Tokens: ['rt', 'mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'amp', 'man', 'always', 'take', 'trash']
```

Fig. 26. Word_tokenize Function Demonstration

This cell shows how to use NLTK's word_tokenize function by taking a sample tweet and tokenizing it, or breaking it down into its individual word tokens. Tokenization is the first step in most NLP preprocessing tasks, as it takes unstructured text data and divides it into smaller units, or tokens, then allows for further analysis, such as classification, or frequency analysis. The original tweet with a subsequent list of tokens from the tweet in printed to demonstrate how the tweet is tokenized.

```
from nltk.tokenize import TweetTokenizer

tweet_tokenizer = TweetTokenizer(preserve_case=False, strip_handles=False, reduce_len=True)

df['tokenized'] = df['tweet'].apply(lambda x: tweet_tokenizer.tokenize(x))
```

Fig. 27. TweetTokenizer Function Demonstration

In this cell, we will be using NLTK's TweetTokenizer. The TweetTokenizer is a NLTK tokenizer that is specifically created for tokenizing tweets. This tokenizer is highly effective for tokenizing tweets since it is capable of handling aspects of tweets including hashtags, mentions handles, repeated character sequences, and emoticons. The tokenizer has been created in this cell with several attributes that the TweetTokenizer has been initialized with including those that convert all text into lowercase letters (preserve_case=False), keep Twitter handles (strip_handles=False), and reduce lengths by tokenizing all repeated characters (reduce_len=True). The tokenizer is applied to subset of the dataset with the most recent ten tweets, and the resultant lists of tokens are added as a new column, tokenized.

```
stop_words = set(stopwords.words('english')) - {'not', 'no'}

def clean_tokens(tokens):
    return [t for t in tokens if (t.isalpha() or t in ['<USER>', '<URL>']) and len(t) > 1
            and t not in stop_words and t != 'rt' and t != 'amp'
]

df['tokenized'] = df['tokenized'].apply(lambda x: clean_tokens(x))
```

Fig. 28. Token Cleaning Function to Remove Stopwords, Punctuation, and Uninformative Tokens

The first cell processes the tokenized tweets and filters out irrelevant or non-informative words. It first defines a custom English stopword list, with 'not' and 'no' included on purpose because of their meaning in sentiment. Second, it defines a function clean_tokens(), which removes tokens that are not alphabetical unless they are USER or URL, are shorter than two characters, are stopwords or are common non-content tokens such as 'rt' for retweet and 'amp' for '&' in HTML . This function is called on each tweet's list of tokens, overwriting the original tokenized column with cleaned versions.

```
test_tweet = df['tweet'][0]

print("Original Tweet:")
print(test_tweet)
print("\nword_tokenize:")
print(word_tokenize(test_tweet))
print("\nCleaned Tokens (TweetTokenizer):")
print(clean_tokens(tweet_tokenizer.tokenize(test_tweet)))

Original Tweet:
rt mayasolovely woman complain cleaning house amp man always take trash

word_tokenize:
['rt', 'mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'amp', 'man', 'always', 'take', 'trash']

Cleaned Tokens (TweetTokenizer):
['mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'man', 'always', 'take', 'trash']
```

Fig. 29. Summarized Outputs of Original Tweet, word_tokenize Function Output, and TweetTokenizer function Output

In contrast, the TweetTokenizer, which is designed to handle social media text, is better at dealing with informal language in text and when the clean_tokens() filter is applied, it will also remove undesired tokens and just retain tokens of interest.

Because of all this, we can conclude that the TweetTokenizer approach is going to be more effective for preprocessing when it comes to tweets since it is returning cleaner and more relevant tokens that downstream task, such as classifying or performing analysis on tweets, can use.

VIII.) Lemmatization

```
from nltk import pos_tag, WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

critical_pronouns = {'us', 'we', 'you', 'i', 'they'}

def get_wordnet_pos(treebank_tag, word=''):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        if word.endswith('ing') or word.endswith('ed'):
            return wordnet.VERB
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

def lemmatize_tokens(tokens):
    tagged_tokens = pos_tag(tokens)
    return [
        word if word in critical_pronouns
        else lemmatizer.lemmatize(word, get_wordnet_pos(pos, word))
        for word, pos in tagged_tokens
    ]

def normalize_informal_words(tokens):
    replacements = {
        'u': 'you',
        'ur': 'your',
        'ya': 'you',
        'r': 'are',
        "ya'll": 'you all',
        'dats': 'that',
        'dat': 'that',
        'dwn': 'down'
    }
    return [replacements.get(word, word) for word in tokens]
```

Fig. 30. Lemmatization and Normalization Functions for POS-Aware Token Processing and Informal Word Replacement

This cell improves on the preprocessing step by adding informal word normalization and lemmatization. The normalize_informal_words() function corrects informal words and abbreviations encountered in tweets such as "u" to "you", "dat" to "that", and "dwn" to "down". Normalization of informal words helps provide a more consistent language before more complicated processing steps are performed. After normalization, the following function is to lemmatize_tokens(), each word will be lemmatized using the tag part-of-speech so that if the word is tagged correctly, the word "running" will be lemmatised as "run". To ensure the correct word pos tagged for lemmatization, a helper function get_wordnet_pos() is used to convert the POS tags into a format that the WordNet lemmatizer will accept. Pronouns "we", "us" and "you" are not altered to maintain their original meaning and context. After the functions are performed, the tokens were normalized and simplified to a more consistent and semantically clear collection of tokens. This is useful for subsequent NLP processing.

```
df['lemmatized'] = df['tokenized'].apply(lemmatize_tokens)

df['normalized'] = df['tokenized'].apply(normalize_informal_words)

df['lemmatized'] = df['normalized'].apply(lemmatize_tokens)
```

Fig. 31. Sequential Application of Informal Word Normalization Followed by Lemmatization on Tokenized Tweets

This cell normalizes informal language and lemmatizes tokenized tweets. First, df['normalized'] = df['tokenized'].apply(normalize_informal_words) iterates through each list of tokens and replaces more informal language examples like "u" to "you" and "dat" to "that" to standardize the language. Second, df['lemmatized'] = df['normalized'].apply(lemmatize_tokens) reduces words to their base word considering what the word meant based on part of speech tagging. Meaning "running" becomes "run" and "houses" becomes "house". In these two steps, the tweets are now less cumbersome due to slang and have a more standard structure that we can use for future analysis.

```
test_index = 0
print("Original:", df['tweet'][test_index])
print("Tokenized:", df['tokenized'][test_index])
print("Normalized:", df['normalized'][test_index])
print("Lemmatized:", df['lemmatized'][test_index])

Original: rt mayasolovely woman complain cleaning house amp man always take trash
Tokenized: ['mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'man', 'always', 'take', 'trash']
Normalized: ['mayasolovely', 'woman', 'complain', 'cleaning', 'house', 'man', 'always', 'take', 'trash']
Lemmatized: ['mayasolovely', 'woman', 'complain', 'clean', 'house', 'man', 'always', 'take', 'trash']
```

Fig. 31. Sample Output Showing Tweet Progression from Original Text to Tokenization, Normalization, and Lemmatization

This output demonstrates how the tweet is processed through each preprocessing stage. The tweet is written in a non-standard structure without punctuation. During tokenization, the tweet is broken into words. Normalized - this step has not changed any of the words in the tweet, since none of the word matches any of the slang terms. During the lemmatization - this processing changed cleaning - to clean, which is the proper way to construct each word in the tweet to give consistency. The series of modifying stages helps to standardize the dataset, as outlined within previous stages of preprocessing and provides the option for future use within NLP models.

IV. Results

This section shows the results of the preprocessing of the dataset, including the transformation of the tweets at every stage of the process, including the "before" and "after" for ease of tracking the data.

I.) Before and After Preprocessed Tweets

In order to showcase the preprocessing pipeline, Table 1 contains the first 5 tweets prior to vs after the cleaning, tokenization, and lemmatization stages. The objective was to show how the raw text was converted to a normalized level suitable for the model.

Raw_tweet	Tweet	Tokenized	Lemmatized
!!! RT @mayasolovely: As a woman you shouldn't complain about cleaning up your house. & as a man you should always take the trash out...	rt mayasolovely woman complain cleaning house amp man always take tras	[mayasolovely, woman, complain, cleaning, house, man, always, take, trash]	[mayasolovely, woman, complain, clean, house, man, always, take, trash]
!!!!! RT @mleew17: boy dats cold...tyga dwn bad for cuffin dat hoe in the 1st place!!	rt mleew boy dats coldtyga dwn bad cuffin dat hoe st place	[mleew, boy, dats, coldtyga, dwn, bad, cuffin, dat, hoe, st, place]	[mleew, boy, that, coldtyga, down, bad, cuffin, that, hoe, st, place]
!!!!!!! RT @UrKindOfBrand Dawg!!!! RT @80sbaby4life: You ever fuck a bitch and she start to cry? You be confused as shit	rt urkindofbrand dawg rt sbabylife ever fuck bitch start cry confused shit	[urkindofbrand, dawg, sbabylife, ever, fuck, bitch, start, cry, confused, shit]	[urkindofbrand, dawg, sbabylife, ever, fuck, bitch, start, cry, confuse, shit]
!!!!!!!!! RT @C_G_Anderson: @viva_based she look like a tranny	rt cganderson vivabased look like tranny	[cganderson, vivabased, look, like, tranny]	[cganderson, vivabased, look, like, tranny]
!!!!!!!!!!!! RT @ShenikaRoberts: The shit you hear about me might be true or it might be faker than the bitch who told it to ya 	rt shenikaroberts shit hear might true might faker bitch told ya	[shenikaroberts, shit, hear, might, true, might, faker, bitch, told, ya]	[shenikaroberts, shit, hear, might, true, might, faker, bitch, told, you]

Table 1. Tweets Data Before and After Preprocessing Steps

```
pd.set_option('display.max_colwidth', None)

df[['raw_tweet', 'tweet', 'tokenized', 'lemmatized', 'class']].head(10)
```

Fig. 32. Table 1 Code

The table shows the first 5 rows of five significant columns from the dataset. Each column captures a processing step in the preprocessing pipeline:

- raw_tweet: Displays the original tweet text as it appeared in the raw dataset, before any processing was applied. This serves as a baseline reference.
- tweet: Displays the cleaned version of the tweet as described above, which includes removing common noise (punctuation, special characters (ex. &, emojis), URLs, mentions, and extra whitespace.)
- tokenized: Displays the cleaned tweet in-tokenized form as individual words or tokens. Here, you will also notice this is structured for the purpose of later NLP methods for frequency analysis or embedding, which will each hypothesize meaning based on the individual words that are assigned.
- lemmatized: Displays the list of tokens in lemmatized form; in other words, each word has been reduced to its base or dictionary form. i.e. "blows" becomes "blow" "bitches" becomes "bitch"; and "girls" becomes "girl".
- class: Displays the annotated label assigned to the tweet as output by the classifier. The three values correspond to:
    - 0 - Hate speech
    - 1 - Offensive language
    - 2 - Neither (neutral/clean content)

II.) Noise Removal Summary

```
patterns = {
    'Usernames (@user)': r'@\w+',
    'URLs (http/https/www)': r'http\S+|www\S+|https\S+',
    'Numbers': r'\b\d+\b',
    'Special Characters': r'[^A-Za-z0-9\s]'
}

noise_counts_after = {label: df['tweet'].str.count(pattern).sum() for label, pattern in patterns.items()}

df['word_count_after'] = df['tweet'].apply(lambda x: len(x.split()))
total_words_after = df['word_count_after'].sum()
noise_counts_after['Total Words'] = total_words_after

plt.figure(figsize=(8,5))
bars = plt.bar(noise_counts_after.keys(), noise_counts_after.values(), color='mediumseagreen')
plt.ylabel('Total Occurrences')
plt.title('Noise Frequency and Total Word Count After Cleaning')
plt.xticks(rotation=25)

for bar in bars:
    height = bar.get_height()
    plt.annotate(f'{int(height):,}',
                 xy=(bar.get_x() + bar.get_width() / 2, height),
                 xytext=(0, 3),
                 textcoords='offset points',
                 ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

Fig. 33. Noise Removal Visualization Code

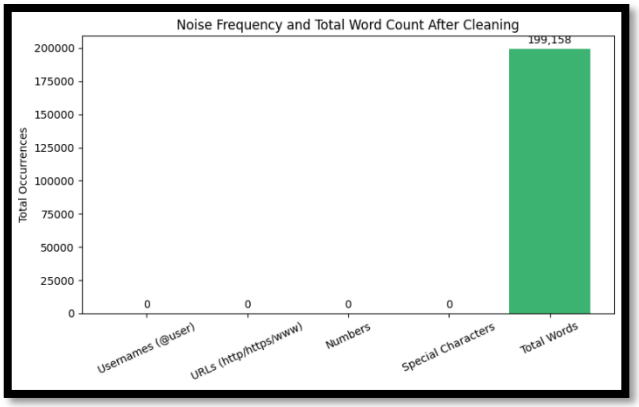

Fig. 34. Noise Removal Summary Bar Graph

To evaluate the performance of the text preprocessing pipeline, we counted the full number of instances of the different types of noise before and after cleaning. The types of noise are:

- Usernames (e.g., @user)
- URLs (e.g., http://, https://, www.)
- Numbers (e.g., dates, quantities)
- Special Characters (e.g., emojis, punctuation marks, or symbols)

Above, the bar chart shows the total counts of data elements remaining after the cleaning step, as well as the total count of the words remaining in the dataset.

This confirms that the preprocessing steps effectively removed all targeted noise types. The dataset now contains only clean, analyzable text suitable for downstream NLP tasks.

III.) Word Count Reduction After Cleaning

```
plt.figure(figsize=(6,4))
bars = plt.bar(['Before Cleaning', 'After Cleaning'], [total_words_before, total_words_after], color=['salmon', 'se
plt.title('Total Word Count Before vs After Cleaning')
plt.ylabel('Word Count')

for bar in bars:
    height = bar.get_height()
    plt.annotate(f'{int(height):,}',
                 xy=(bar.get_x() + bar.get_width() / 2, height),
                 xytext=(0, 3),
                 textcoords='offset points',
                 ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

Fig. 35. Word Count Comparison Code

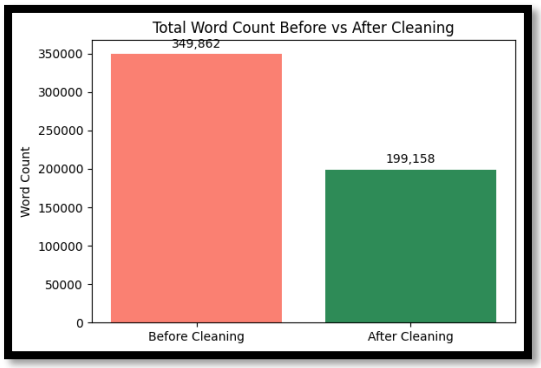

Fig. 36. Total Word Count Before vs After Text Cleaning Graph

Fig. 35 produces a bar chart that visually shows the total words in the dataset with before and after text cleaning.

The total number of words in the dataset was greatly reduced from the original word count after the complete text preprocessing. In the chart, the word count decreased from 349,862 before cleaning to 199,158 after cleaning. Such a reduction is entirely reasonable given the fact that noise was removed, including usernames, URLs, numbers, special characters, and other language artifacts that were not needed for the current modelling task. The chart alone clearly indicates the significant effectiveness of the preprocessing pipeline applied in reducing the dataset to linguistically meaningful content.

V. DISCUSSION

I.) Interpretation of Results and Significance

The results of the preprocessing pipeline were quite significant: the size of the dataset shrank considerably to 199,158 words from a total of 349,862 words, which indicates a considerable portion of the dataset was noise. Additionally, the number of tokens removed for usernames (19,287), URLs (3,158), numbers (18,366), and special characters (157,341) clearly indicates much of the noise was related to the text. The reduced and combined sections of cleansed data clearly indicate the relatively noisy nature of social media text, with particular reference to Twitter.

The reduction of tokens from the noisy data provides a reduction in noise to the dataset, improving overall quality for downstream natural language processing tasks. Providing improved, cleansed, and standardized input makes it far easier for the machine to tokenize, with less document feature sparsity and improved model learning and performance.

II.) Impact of Each Preprocessing Step

Every preprocessing step contributed to the transformation of the raw, noisy tweets into a cleaner and more uniform dataset for natural language processing. Below are the respective techniques implemented and their observed outcomes:

- Lowercase: This normalized the text to a common case, which eliminated concerns regarding a case-sensitive word being tokenized as different terms.

- Contraction Expansion: This helped improve clarity and allowed better lexical matching of base forms.

- Username, URL, Number, and Special Character Removal: These steps help remove non-semantic content adding a high degree of noise to the dataset. Usernames, URLs, and numeric values were eliminated because they added little-to-no relevance to the identification of hate or obscene languages. Special characters, also emojis, or excessive punctuation were also removed so that no considerable confounding could arise in tokenization. This all made the texts much clearer and more consistent.

- Stopwords Removal: High-frequency low-information words were removed, thereby helping reduce dimensionality for machine learning applications.

- Token Normalization: Foreign elongated words like "loooool" or "yessss" were handled by shrinking the multiple instances of a character to a standard base form ("lol," "yes") to be consistent with vocabulary.

III.) New Noise Identified and Custom Preprocessing Solutions

One unusual discovery while preprocessing was the occurrence of repeated single-character tokens like "a a a a" and long forms of rigor like "loooove" or "goooooood." While not formally emphasized in the lecture content, repeated-character phenomena were something found commonly in casual online text, as mentioned, I took note, and it was found often in the tweet data.

To solve this issue the relevant characters were handled with the implemented cleaning rules. Regex was used to format the repeated letters. Although, I filtered single-character spam in tokenization to not skew word frequency distributions.

These custom preprocessing tasks were a great improvement to semantic consistency and noise reduction in the dataset.

IV.) Limitations and Trade-Offs

Although the preprocessing steps yielded overall improvements, there were some relevant compromises or trade-offs to consider:

- Potential Over-cleaning: Emojis and prolonged text can suggest a strong sentiment that could be useful for nuanced sentiment analysis but had been remodifying the dataset.

- Loss of Informal Markers: Twitter-based markers such as hashtags may provide a thematic cue, such as #BlackLivesMatter, but were lost again while suffering through the noise removal stage.

- Assumptions during Normalization: Repeated characters and punctuation may not always be noise; in some contexts, they might be more participatory/expressive or semantically relevant.

- No POS-tagging or Lemmatization: The analysis parameters did not stretch to the more advanced

preprocessing steps that would have further increased generalizability/robustness.

VI. Conclusion

The exercise illustrated how important preprocessing is for organizing noisy real-world data for natural language processing tasks. The dataset was processed with a battery of preprocessing methods, including case normalization, contraction expansion, noise removal, stopword removal, and token normalization, changing the dataset from a messy and inconsistent state to a cleaner, somewhat standard state. During preprocessing, the size of the dataset was also reduced from 349,862 words to 199,158 words. This difference highlighted the amount of non-informative data that is often found in social media datasets.

The activity demonstrated how preprocessing directly impacts the quality of downstream tasks, such as tokenization, classification, and model training. It also pointed to compromises and challenges to incorporate elements from informal language that are semantically relevant, such as hashtags and emojis, while the positive aspects of preprocessing, such as less feature space sparsity and more semantic consistency, made consideration of these compromises irrelevant.

More importantly, in completing this task, we had the real experience of recognizing actual noise in real time and how we could develop a custom approach to cleaning data to eliminate many forms of creaky, dirty, or inconsistent aspects of the text data, for example, dealing with exceptionally long tokens and long strings of repeated characters, and perhaps most importantly, this emphasized the value of designing preprocessing to suit the context of the data as well as the purpose of the task.

The influence of different preprocessing techniques, including lemmatization or part-of-speech tagging, and their effect on model performance might be the focus of future research. Emojis and hashtags are examples of informal characteristics that may be retained and analyzed to assist sentiment and topic modeling tasks. A key step towards developing more complex NLP pipelines and comprehending their consequences for practical machine learning is this exercise.

VII. References

[1] C. Stryker and J. Holdsworth, “What is Natural Language Processing?”, IBM, 2023. [Online]. Available: https://www.ibm.com/think/topics/natural-language-processing. [Accessed: Aug. 3, 2025].

[2] CodeSignal, "Lowercasing Text for Uniformity in NLP," CodeSignal, 2023. [Online]. Available: https://codesignal.com/learn/courses/text-data-preprocessing-in-python/lessons/lowercasing-text-for-uniformity-in-nlp. [Accessed: Aug. 3, 2025].

[3] GeeksforGeeks, "Expand Contractions in Text Processing," GeeksforGeeks, 2023. [Online]. Available: https://www.geeksforgeeks.org/nlp-expand-contractions-in-text-processing/. [Accessed: Aug 3, 2025].

[4] T. Mistry, "Removing Punctuation and Special Characters," Medium, 2023. [Online]. Available: https://medium.com/@mistrytejasm/text-preprocessing-removing-punctuation-and-special-characters-e3de4cece082. [Accessed: Aug 3, 2025].

[5] Analytics Vidhya, "How to Remove Stopwords for Text Normalization," Analytics Vidhya, 2019. [Online]. Available: https://www.analyticsvidhya.com/blog/2019/08/how-to-remove-stopwords-text-normalization-nltk-spacy-gensim-python/#h-why-do-we-need-to-remove-stopwords. [Accessed: Aug. 3, 2025].

[6] Analytics Vidhya, "How to Get Started with NLP: 6 Unique Ways to Perform Tokenization," Analytics Vidhya, 2019. [Online]. Available: https://www.analyticsvidhya.com/blog/2019/07/how-get-started-nlp-6-unique-ways-perform-tokenization. [Accessed: Aug. 3, 2025].

[7] Analytics Vidhya, "Stemming vs Lemmatization in NLP: Must-Know Differences," Analytics Vidhya, 2022. [Online]. Available: https://www.analyticsvidhya.com/blog/2022/06/stemming-vs-lemmatization-in-nlp-must-know-differences. [Accessed: Aug. 3, 2025].

[8] T. Davidson, “hate-speech-and-offensive-language,” [Online]. GitHub. Available: https://github.com/t-davidson/hate-speech-and-offensive-language. [Accessed: Aug. 3, 2025].